前置知識與本章目標

- 前置知識為指標、物件生命週期、Stack/Queue ADT 與第 1 章不變量。
- 能區分節點位置、節點值與節點所有權，並說明哪個操作真的需要搜尋。
- 能實作與追蹤單向、環狀、雙向串列，以及鏈結堆疊、佇列與多項式相加。

單向串列的合法狀態

- 節點包含值與 `next`，`head` 指向第一個節點。空串列使用 null。
- 不變量為從 `head` 沿 `next` 形成有限無環鏈，最後指向 null，每個節點由唯一容器負責釋放。
- 若保留 `tail`，非空時它必須指向最後節點且 `tail.next=null`；空時 `head`、`tail` 同為 null。
- 外部指標可作非擁有參照，但被指向節點刪除後即失效。有效位址不等於有效物件生命週期。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

已知前驅時的插入

在 `p` 後插入新節點 `x`。

1. 配置並建構 `x`，令 `x.next=p.next`。
2. 令 `p.next=x`。若 `p` 原先是尾節點，更新 `tail=x`。
3. 更新元素數。若建構失敗，原串列仍保留。

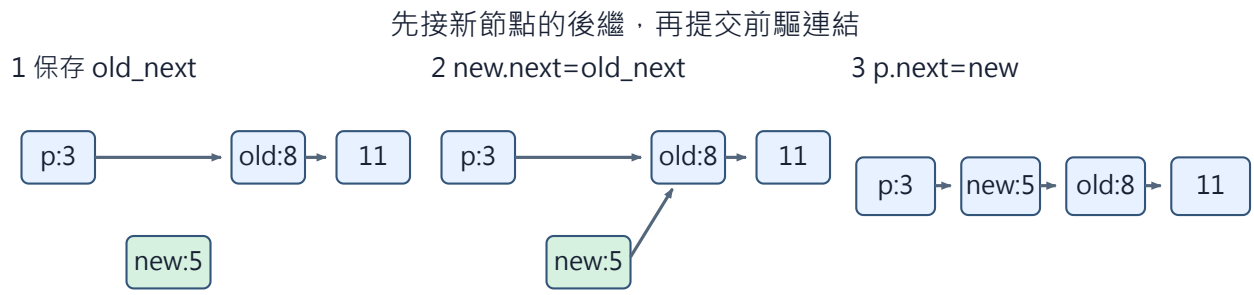
此操作在已知合法前驅、配置成本視為常數時為 `O(1)`。若先用索引找第 `i` 個前驅，搜尋成本為 `O(i)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單向串列插刪的完整所有權轉移

```
insert_after(p,value):
    require p is a live node of this container
    old_next=p.next
    x=allocate_and_construct(value)
    x.next=old_next
    p.next=x
    if tail==p: tail=x
    size+=1
erase_after(p):
    require p is live and p.next!=null
    victim=p.next; after=victim.next
    p.next=after
    if tail==victim: tail=p
    destroy_and_release(victim); size-=1
```


單向串列插刪的完整所有權轉移 （續）



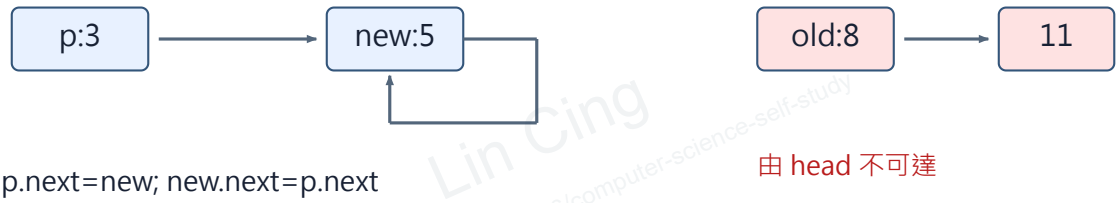
對p值3、後繼8→11插入5。配置後x未從head可達；設定x.next=8後，舊鏈仍3→8→11且新節點也能到8；最後p.next=x提交為3→5→8→11。若倒過來先p.next=x再x.next=p.next，會得到x.next=x自環，8與11失去從head的路徑。這是明確可達性破壞，不只結果值排列錯誤。

頭插／頭刪需獨立處理head，或用不含資料的dummy前驅統一操作。若dummy成為最後剩餘節點，對外空狀態應head=null,tail=null，不把dummy當使用者資料。已知前驅的連結更新O(1)；外部任意pointer不滿足此契約，依位置搜尋前驅仍O(n)。

錯序插入的可達性反例

```
old_next=p.next
p.next=x              // wrong first update
x.next=p.next         // reads x, not old_next
```

錯序造成 new 自環，失去舊後綴



p.next=new; new.next=p.next

由 head 不可達

原head=p且鏈3→8→11。錯序執行後由head可走3→5→5→…；原8與11仍配置在記憶體卻失去容器的擁有路徑，形成遺漏與可能洩漏。即使某個外部變數仍持有8的位址，也不代表容器表示有效。正確順序是先以已保存old_next接x，再提交p.next=x。若多執行緒可同時觀察，這兩個寫入也不自動成為原子交易；本章契約限定單執行緒結構更新。

刪除必須先保存下一個位置

- 刪除 `p` 後的節點時，先保存 `victim=p.next` 與 `after=victim.next`，再令 `p.next=after`，最後釋放 victim。
- 若 victim 是尾節點，更新 `tail=p`。刪除唯一節點則須同時使 head、tail 為空。
- 解參照 victim 必須發生在釋放前。釋放後再讀取 `victim.next` 是 use-after-free。
- 只有待刪節點指標、沒有前驅時，一般無法在單向串列 `O(1)` 刪除尾節點。「複製下一節點的值」也會改變節點身分語義。

頭端、佇列端點與迭代清除

```
push_front(v):
    x=Node(v,next=head); head=x
    if tail==null: tail=x
    size+=1
pop_front():
    require head!=null
    x=head; result=copy(x.value); head=x.next
    if head==null: tail=null
    release(x); size-=1; return result
enqueue(v):
    x=Node(v,next=null)
    if tail==null: head=x
    else: tail.next=x
    tail=x; size+=1
clear():
    while head!=null:
        x=head; head=x.next; release(x)
    tail=null; size=0
```


頭端、佇列端點與迭代清除 （續）

這四個操作共用同一個擁有節點的單向表示，鏈結堆疊以 push_front/pop_front 操作，鏈結佇列以 enqueue/pop_front 操作。前提為單執行緒、節點配置與值複製成功；若複製 result 會拋出例外，必須在修改連結前完成，失敗時原串列保留。每次釋放前先保存下一個節點，不能讀取已釋放的 x.next。

從空狀態 (head,tail,size)=(null,null,0) 入列4，兩端同指4且 size=1；入列9得4→9、tail=9、size=2；出列4得 head=tail=9、size=1；再出列9得到兩端 null、size=0；再入列7可安全建立新鏈。若只清 head 不清 tail，最後一步將沿已釋放尾節點寫入。單點與空鏈是表示契約的一部分。

歸納檢查：新節點只由原尾或 head 連入一次，無環且末端 next=null；刪頭只移除第一點，剩餘次序與所有權不變；size 與可達節點數同步。每項端點操作在固定大小值和常數配置模型下最壞 O(1)。clear 每輪恰釋放一個可達節點，時間 Θ(n)、額外堆疊 O(1)，可避免逐節點遞迴析構的線性呼叫深度。

三指標反轉的不變量

初始 prev=null,curr=head，重複保存 next=curr.next、令 curr.next=prev，再前移 prev,curr。

- 不變量為 prev 指向已反轉前綴，curr 指向尚未處理後綴，兩者節點集合互斥且聯集為原串列。
- 原串列 3→8→11 經一輪成 3→null，未處理部分為 8→11。
- 終止時 curr=null，令 head=prev。若維護 tail，舊 head 成為新 tail。
- 每個節點處理一次，時間 O(n)、輔助指標空間 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

反轉的完整程序與節點分割不變量

```
reverse(head):
    old_head=head; prev=null; curr=head
    while curr!=null:
        after=curr.next; curr.next=prev
        prev=curr; curr=after
    head=prev; tail=old_head
```

反轉3→8→11的(prev,curr)依序為(空,3→8→11)、(3,8→11)、(8→3,11)、(11→8→3,空)。兩段互斥且覆蓋原節點，是保留全部資料的證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐步反轉鏈結串列

單向串列反轉：兩段互斥，合起來保有所有節點

原始：3 → 8 → 11 → null

輪次	prev：已反轉前綴	curr：未處理後綴
0	null	3 → 8 → 11 → null
1	3 → null	8 → 11 → null
2	8 → 3 → null	11 → null
3	11 → 8 → 3 → null	null

每輪先保存 curr.next，再反轉連結；完成後 head=prev，tail=原 head。

空鏈和單節點也適用；圖中的數字表示節點身分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ 所有權與複製語義

- 用 `unique_ptr` 表示單一所有權，可以讓節點釋放責任隨所有權轉移。非擁有 tail 可用一般指標但必須維持有效性。
- 含 owning raw pointer 的類別若採預設淺複製，兩個容器可能重複釋放同一節點。必須定義深複製或明確禁用複製。
- `shared_ptr` 不是所有鏈結結構的預設答案。雙向或環狀 owning links 會形成參照計數循環。
- 長鏈的遞迴析構可能造成很深的呼叫堆疊；授課實作可採迭代清除，並說明生命週期處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鏈結堆疊與佇列

結構	指標安排	更新成本
堆疊	head 為頂端	head 插入、刪除各 O(1)
佇列	head 出列、tail 入列	保留 tail 時各 O(1)

- 鏈結佇列從空狀態 enqueue 時，head、tail 都指向新節點。
- dequeue 最後一個節點時，必須把 tail 也清成 null，否則下一次入列可能沿懸空指標寫入。
- 操作成本不包含任意大小值的複製。配置器的真實延遲也不由 ADT 保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式的正規表示

- 每個節點存 `(exponent,coefficient)`，指數為非負整數，按指數嚴格遞減，係數非零。
- $P=4x^5-2x^2+7$ 表示為 `(5,4)→(2,-2)→(0,7)`。
- 同指數多次出現屬於未正規化輸入。需合併係數，係數抵消成零時移除節點。
- 零多項式用空串列表示，不把 `(0,0)` 與空串列混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式相加是有序合併

- 令 $Q=-4x^5+3x^3+2x^2$。兩游標從最高指數開始，比較目前指數。
- 指數較大的一項直接輸出並前移。指數相同則相加，僅在係數非零時輸出，再同時前移。
- 本例五次項與二次項抵消，得到 $P+Q=3x^3+7$。
- 不變量為輸出已正規化，且等於兩輸入已消耗部分的和。每個節點最多前進一次，時間 $\Theta(p+q)$，輸出空間 $O(p+q)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式合併的逐項消耗

```
add_polynomials(p,q):
    require both normalized, descending exponents
    out=[]
    while p!=end and q!=end:
        if p.exp>q.exp: append_copy(out,p); advance(p)
        else if q.exp>p.exp: append_copy(out,q); advance(q)
        else:
            c=p.coeff+q.coeff
            if c!=0: append(out,(p.exp,c))
            advance(p); advance(q)
    append_copy(out,remaining(p)); append_copy(out,remaining(q))
    return out
```

同一$P=4x^5-2x^2+7$與$Q=-4x^5+3x^3+2x^2$：先比較5與5，係數$4+(-4)=0$不輸出；再比較2與3，輸出$3x^3$；再比較2與2，$-2+2=0$不輸出；Q結束後輸出剩餘常數7，結果$3x^3+7$。游標位置每次至少前進一個，輸出嚴格遞減且無零係數。

若係數採固定整數，必須指定溢位如何處理；若採浮點數，近零合併的容差不是精確代數等於零，會改變抽象值。程式應先選精確整數、有理數或明確的近似模型，不能以浮點容差悄悄宣稱代數恆等。

環狀串列的終止條件

- 非空環狀單向串列滿足 `tail.next=head`，不以 null 終止。走訪應在回到起點時停止。
- 保留 tail 時，head 為 `tail.next`。在頭尾插入均可改動固定數目的 links。
- 原例 `A→B→C→A`，從 B 開始走訪一圈應得到 B、C、A，而不是永遠等待 null。
- 若採 sentinel，空串列可用 sentinel 指回自身。sentinel 是結構節點，不屬於使用者資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀串列的尾插與端點成本

```
circular_push_back(value):
    x=new_node(value)
    if tail==null: x.next=x; tail=x
    else: x.next=tail.next; tail.next=x; tail=x
```

環狀空鏈加入A後A.next=A；再加入B後tail=B、B.next=A、A.next=B。走訪用do/while在回到head時停止，空鏈必先檢查tail=null。只保留tail時刪頭O(1)，刪尾須找前驅O(n)，不能因頭尾插入快就宣稱全部端點操作都快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀佇列：唯一節點出列要清空 tail

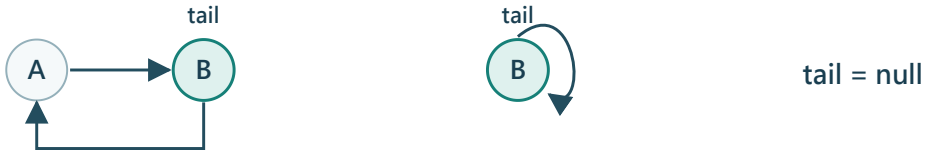
只保留 `tail`，非空時首節點為 `tail.next`。若 `tail.next==tail`，代表只有一個節點；出列並釋放它之前，必須把 `tail` 設為 null。一般情況則把 `tail.next` 改為舊首節點的後繼，再釋放舊首節點。

```
dequeue():
    if tail == null: return empty
    victim = tail.next
    value = copy(victim.value)
    if victim == tail: tail = null
    else: tail.next = victim.next
    destroy_and_release(victim)
    return value
```

例：A→B→C→A 且 tail=C，出列後為 B→C→B，tail 仍是 C；若原來只有 A→A，出列後 tail 必須為 null。假設值複製、釋放與回傳均成功；泛型值若可能拋例外，需另訂完整例外契約。

環狀佇列：最後節點出列

環狀佇列：刪除最後節點的前後



① 原狀 A→B→A　② 出列 A：B→B　③ 出列 B：空

victim=tail.next；只有一個節點時，先將 tail 設為 null，再釋放 victim。

之後入列 C，會建立 C→C；不可使用已釋放的 B。

粗框標示尾節點；空狀態沒有有效的 tail。

Floyd 快慢指標：找到環或證明走到底

兩指標都從 head 開始。每輪先確認 fast 與 fast.next 皆非 null，再令 slow 走一步、fast 走兩步；若相遇，結構有環。若檢查失敗，已到達無環鏈的末端。此處假設節點在演算法執行期間保持有效，沒有並行修改。

```
slow=head; fast=head
while fast != null and fast.next != null:
    slow=slow.next; fast=fast.next.next
    if slow==fast: return meeting
return no_cycle
```

空串列、單節點末端為 null 都回報無環；單節點指向自己則第一輪相遇。演算法不改動節點連結。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼再次相遇恰在環入口？

設 head 到環入口有 μ 條邊、環長 $\lambda \geq 1$。第一次相遇發生在環內；設 slow 已走 t 步，fast 走 $2t$ 步，兩者在環上的位置相同，因此 $t \equiv 0 \pmod{\lambda}$。

令一個指標回 head，另一個留在相遇點，兩者每輪各走一步。走 μ 步後，前者抵達入口；後者的位置是 $(t - \mu + \mu) \bmod \lambda = 0$，也在入口。在這之前，從 head 出發的指標尚未進環，不可能與環內指標相遇。因此第二次相遇恰是入口。

從 head 到首次相遇、再到入口，時間 $O(\mu + \lambda)$，額外指標空間 $O(1)$。無環長度為 n 時，時間 $O(n)$；兩種情況都不依賴節點值是否重複。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

數值追蹤：前綴長 2、環長 3

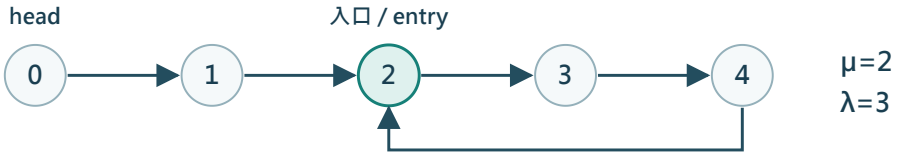
指標鏈為 `0→1→2→3→4→2`，所以 $\mu = 2$、$\lambda = 3$。第一次相遇前，各輪 `(slow,fast)` 為 `(1,2)`、`(2,4)`、`(3,3)`；回 head 後，兩指標依序為 `(0,3)`、`(1,4)`、`(2,2)`，入口是 2。

不可只因「兩個節點的值相同」就判為相遇：比較的是節點身分。找環入口與刪除環是不同操作；本演算法只回傳入口，不修改 `next`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快慢指標的兩次相遇

Floyd：第一次相遇，再尋找環入口



第一階段：每輪 slow 一步，fast 兩步

(1,2) → (2,4) → (3,3)

重設一個指標到 head；兩者都每輪一步

(0,3) → (1,4) → (2,2)

比較節點身分；兩次相遇的節點不一定相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向串列的局部一致性

- 每個節點有 prev、next，鄰接節點須滿足 `x.next.prev=x` 及 `x.prev.next=x`，端點按 sentinel 策略處理。
- 在相鄰節點 p、q 間插入 x，設定 `x.prev=p,x.next=q,p.next=x,q.prev=x`。
- 刪除已知節點 x，讓 `x.prev.next=x.next`、`x.next.prev=x.prev`，再釋放 x。
- 若節點已知，更新為 `O(1)`。依值找節點仍是 `O(n)`，不能把「移除已知節點」偷換成「搜尋並移除」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向串列刪除與參照生命週期

```
doubly_erase(x):
    require x is a live owned data node, x!=H and x!=T
    p=x.prev; q=x.next
    p.next=q; q.prev=p
    release(x); size-=1
```

本段固定採兩個不含資料的 sentinel H、T。H.prev=null、T.next=null；空串列為 H.next=T、T.prev=H，size=0。每個資料節點的 prev、next 均非 null，因此以下 p.next 與 q.prev 可安全解參照。若改採無 sentinel 的 null 端點，刪首／尾節點必須另更新 head／tail，不能直接套此程式。

只有一筆資料時 H↔x↔T，刪除取 p=H、q=T，接回 H.next=T、T.prev=H，size 由 1 降 0；H、T 保留且絕不釋放。刪多筆串列的第一或最後資料同樣由鄰接 sentinel 接回。合法節點 handle 與所屬容器的確認在進入指標操作前完成。

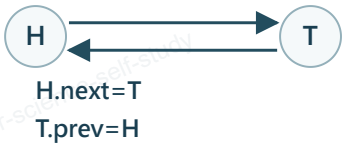
雙向鏈p↔x↔q刪除後p.next=q、q.prev=p，故兩個互反條件同時恢復。剩餘節點的外部參照保持有效，x的所有參照失效；不能只改一側，否則反向與正向走訪會看到不同序列。

雙向串列：刪除唯一節點

雙向 sentinel：刪除唯一資料節點



① 保存 p=x.prev=H、q=x.next=T
② 設 p.next=q；設 q.prev=p
③ 釋放 x；size 由 1 變 0



H、T 保持有效；x 的外部指標不得再解參照。

串列與陣列的選擇

操作與限制	動態陣列	單向串列
依索引讀取	O(1)	O(n)
已知前驅插入	O(n) 搬移	O(1) links
連續掃描	通常有儲存局部性	需追蹤指標
額外每項空間	容量閒置	next 與配置開銷

只有完整工作負載才能決定取捨。沒有實測時，不能以 links 數直接宣稱實際較快。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 forward_list、list 與 unique_ptr。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study